

Agent Harness Mass Brainstorm Doc

OVERVIEW

Problem: Northwell teams have access to large Epic datasets and 40,000+ HTML table documentation files. Due to migrations, users struggle to identify relevant tables, columns, joins, and safe query patterns. Naively connecting an LLM to BigQuery creates risks: SQL injection, destructive queries, high cost, PHI exposure, hallucinated schemas, and uncontrolled tool use.

Goal: Design a safe agent to convert a user’s natural-language questions about Northwell/Epic data into SQL/Python queries and return the desired information, while minimizing the agent’s permissions, actions, and costs. MVP task should be read-only “data-spelunking.”

Target Audience:

Persona	Need	Example question
Data Scientists	Explore correct Epic tables/columns after migration	“Where can I find encounter diagnosis fields?”
Data engineer/analyst	Validate joins and safe query patterns	“How do I join encounters to departments?”
Clinical operations/informatics analyst	Run aggregate operational questions	“How many encounters by department last month?”
Data governance reviewer	Ensure safe use of PHI and BigQuery	“What policy gates block patient-level output?”

Features:

TECH STACK:

Python

Primary implementation language for the harness, retrieval, LLM calls, BigQuery integration, evals, and UI.

LangGraph

Use this for the explicit computational graph: input gate → retrieval → query plan AST → SQL generation → validation → dry run → execution → interpretation. LangGraph is designed around workflows, agents, routing, structured outputs, and graph state.

Pydantic

Use for typed objects at each node: QuestionIntent, QueryPlan, SQLCandidate, GateDecision, ExecutionPlan. This makes LLM outputs schema-validated instead of free-form text.

BeautifulSoup

Use to parse the 40,000 Epic HTML documents into structured table cards, column cards, join hints, and citations.

Hybrid retrieval: BM25 + vector search

Use lexical search for exact Epic table/column names and vector search for semantic questions like “discharge disposition” or “patient movement history.” (Possibly use Whoose, Lucene, OpenSearch)

Embeddings

text-embedding-3-large for semantic retrieval

Vector Database: Qdrant or Chroma

Store embeddings for documentation

FAISS / pgvector / Vertex AI Vector Search (Not sure which one suits needs best yet)

Use one approved vector store to index table cards and column descriptions.

BigQuery Python Client

Use for metadata inspection, dry runs, and approved read-only query execution. BigQuery dry runs estimate bytes before execution, and maximumBytesBilled gives a hard execution cap.

SQLGlot

Use as the SQL parser/static validator. It supports the BigQuery dialect and lets you inspect the SQL AST for forbidden statements, unknown tables, SELECT *, multi-statements, etc.

FastAPI

Use for the backend API: submit question, run graph, fetch audit trail, return SQL/result/citations.

Streamlit

Useful for the demo UI.

SQLite/Postgres/BigQuery audit table

Store every run: prompt hash, retrieved docs, query plan, generated SQL, validation decisions, dry-run bytes, BigQuery job ID, result shape, and final answer.

Anthropic SDK

Northwell’s approved AI environment. The Anthropic SDK supports agents, tools, guardrails, handoffs, structured outputs, and tracing-style agent runs.

MCP, possibly optional for MVP

Design tools in an MCP-shaped way, but maybe not require MCP. MCP is useful later as the standardized boundary for safe tools like search_docs, get_schema, dry_run_sql, and execute_approved_query. MCP’s architecture is host/client/server-based and exposes tools, resources, and prompts through typed protocol primitives.

BRAINSTORMING

Allowed Tools	Prohibited Tools
Claude SDK: • File operation: Read • Search: Glob and Grep • Discovery: ToolSearch • Orchestration: AskUserQuestion, TaskUpdate Other Tools • The agent may generate SQL. • search_docs(query): find relevant documents to parse • get_table_schema(table): get descriptions of table and its columns • check_sql(SQL): checks to make sure the SQL has no blocked (list	Claude SDK: • File operations: Edit/Write • It may not generate arbitrary executable Python. • Execution: Bash (? revisit this - helpful but need to restrict commands) • Orchestration: Agent (no subagents), TaskCreate(shouldn’t be able to make its own task) SQL: • DROP - DELETE - UPDATE - MERGE - INSERT

below), read-only, • estimate_cost: • run_query(sql): runs only validated queries • summarize(result); summarizes to user	CREATE CREATE OR REPLACE ALTER TRUNCATE EXPORT DATA CALL EXECUTE IMMEDIATE DECLARE BEGIN COMMIT ROLLBACK • SQL validation is AST-based (SQLglot), not regex-only. Regex may be used as a fast pre-check, but execution requires parser-level validation. • multi-statement scripts semicolon-separated SQL remote functions external connections temporary function creation unapproved wildcard table scans SELECT * queries against non-allowlisted datasets Other • no internet access • CANNOT change permissions • Cannot create new agents • Cannot access secrets
--	--

Turns and Budget

- max_turns: (limit here)

- max_budget_usd: (valuable, esp if running in parallel)

Brainstorm layers:

- Input/Question
- *HOOK: see if the question is allowed (non destructive) and doesn't ask for sensitive info (PII) directly (ex which patient was ___ at this date and time___ is Not allowed)*
- Agent
- *HOOK: restrict retrieval to allowed columns only*
- RAG - reference database + html docs, retrieve relevant tables and column descriptions from epic data
- *HOOK: Check relevancy of retrieved data, if it is good enough so LLM can create with enough context*
- LLM creates code + cost to run estimation
- *HOOK: review LLM's code to make sure it doesn't start anything prohibited (ex it can only be read only, can't access files, cannot spawn other agents, cant run commands)*
- Safety check / Permission validation
- *HOOK: check estimated cost, timeout, permissions before running*
- Execute
- *HOOK: check the returns for any prohibited responses (i.e. PII if not allowed)*
- Return

Retrieval Layer:

- NL2SQL research shows that a semantic layer improves retrieval by adding an intermediate representation between the LLM and the database: [✗ A Semantic-Layer-Mediated Agent for Natural Language to SQL over...](#)
- Some search may require exact word matching, so vector search alone is infeasible. Better structure would be lexical search + vector search + maybe table relationships/edges

SQL Gate Layer:

Evaluation:

THREAT/RISK MATRIX

Risk	Failure mode	MVP mitigation
SQL injection	User input changes SQL structure	Parameterize values; identifiers come only from catalog allowlist.
Destructive SQL	Model emits DROP , DELETE , CREATE , TRUNCATE	Parse SQL AST; allow only SELECT / WITH ; service account has no write permissions.
Excessive BigQuery cost	Full table scan over large Epic table	Dry run required; cost threshold; maximum_bytes_billed ; block SELECT * ; require partition filters.
LIMIT misuse	Query uses LIMIT 10 but still scans terabytes	Cost gate uses dry-run bytes, not returned row count.
Raw PHI exposure	Patient-level rows shown to user or LLM	Aggregate-only MVP; block sensitive columns; row cap; result gate.
Prompt injection	User tells model to bypass policy	Policies enforced in code, not prompts.
Indirect prompt injection	HTML doc contains instruction-like text	Retrieved docs are untrusted context; cannot modify policy.
Hallucinated schema	Model invents nonexistent table/column	Validate all identifiers against catalog + BigQuery schema.
Bad joins	Model joins on wrong keys	Prefer join hints from docs/catalog; flag low-confidence joins.

Python execution abuse	Model runs arbitrary Python	No Python execution tool in MVP.
Bash/process spawn	Model invokes shell or subprocess	No shell/subprocess tools registered; no stdio MCP process spawning in V1.
Multi-agent recursion	Agent spawns agents or loops indefinitely	Fixed graph; max steps; no dynamic tool registration; no subagent spawning.
Misleading interpretation	SQL correct but summary wrong	Show SQL, result shape, assumptions, caveats, and source citations.
Unauthorized table access	Model queries non-approved dataset	Dataset/table allowlist; Least-privilege service account; execution gate.
Audit gap	Query runs with no trace	Log every gate decision, SQL hash, dry-run bytes, job ID, and final output.

NODES:

Node	Details
1. User Prompt/Question	Purpose: Receive natural language question. Input: Human question about data. Output: User request passed into input gate

2. Input/ Policy Gate	Purpose: Check if the request is allowed. • Reject destructive requests. • Reject direct requests for PII or PHI. • Reject requests asking for individual info. • Allow analytic read-only prompts. Ex: Allowed: ‘How many patients were admitted to __ department in June?’ Not allowed: ‘Which patient(s) was admitted to __ department on June 10th?’
3. Intent Classifier	Purpose: classify the type of question the user is asking Ex: Summary statistics (count, average, min, max), overall trend, comparisons, cohort definitions, unknown (needs more info from user)
4. IF NECESSARY: Clarification	Purpose: request more information from user if prompt is ambiguous • Loop back to intent classifier before continuing to #5
5. Retrieval permission gate	Purpose: restrict what agent is allowed to retrieve • Can only retrieve approved epic tables, allowed columns, possibly exclude sensitive keywords depending on final task/goal
6. Retrieval Layer (RAG), 40k HTML	Purpose: Retrieve required context to generate query

	Output: Retrieval should be treated as untrustworthy/un-verified context Components: • Lexical Retrieval: exact table names, column names, abbreviations, terminology • Vector Retrieval: semantic similarity • Graph retrieval: Table relationships “what tables join to _____” • Semantic layer: adding an intermediate representation between the LLM and the database map clinical concepts add business meaning to terminology • Sources: Epic HTML table and column descriptions. • Output: Ranked table candidates, candidate columns, join paths, business definitions, supporting documentation
7. Retrieved Context Gate	Purpose: Validate enough context was retrieved to generate a query. • Check retrieved table relevancy columns present, are relevant, enough information • Check if join paths are available • If insufficient, retrieve additional context or clarification. • Loop back to #6, then pass through this gate before #8

8. Query Plan (AST)	Purpose: convert the request into a structured execution plan before code generation. Output: Pre-validated, raw plan output Detailed layout: • Target metric • Tables • Columns • Filters • Join relationships • Aggregations • Time constraints • Groupings • Expected output
9. Plan Safety Gate	Purpose: Validate the plan before code generation. • Check approved tables and columns only. • No unnecessary PII/PHI/sensitive fields. • Reasonable computational complexity (reasonable cost)
10. SQL/Python Generation	Purpose: Generate executable SQL or python code from approved query plan Restrictions: • Read-only operations • No filesystem access • No shell commands • No agent spawning • No network access • No database writes

	• No secrets access Output: Generated SQL/Python
11. SQL Static Validation	Purpose: Verify the generated code before execution Check: NO: DELETE UPDATE INSERT DROP ALTER MERGE TRUNCATE – see threat risk matrix for more • No forbidden tables/column access • No unsafe SQL functions (see risk matrix). • Query matches approved execution plan purpose. • If it fails: repair SQL via approved query plan and check again
12. BigQuery Dry Run	Purpose: Estimate execution cost without running query. Output: bytes, est runtime/cost, tables accessed, query validity
13. Cost + Execution gate	Purpose: Perform final check before execution. Check: • estimated cost below limit • acceptable runtime • Read-only execution

	If it fails: revise query from #8 and continue through path through Cost/Execution gate
14. Read-Only Query Execution	Purpose: Execute the approved query. Restrictions: • Read-only. • Enforce timeout and other limits. • Don't expose secrets to LLM Output: Raw result
15. Result Safety Gate:	Purpose: Validate results before returning. Check: • no PHI/PII leaking. • No identifiers. • Results match approved request
16. Interpretations & Citations	Purpose: Convert raw result into interpretable results • Plain language explanation • Tables • Documentation references • Limitations • Assumptions____etc

17. Bounded Follow Up Loop	Purpose: Allow limited refinement - prevent uncontrolled execution • Maximum reruns - LIMIT • No injecting new maximum rerun • No spawning new agent • Pass every follow up through validation • Start at the latest correct node and retry within limit (don't loop forever), and don't give new loop any extra permissions • Should oversee entire pipeline
-----------------------------------	--

EDGES

Tools	Safety Checks
Edge 1: User Prompt → Input / Policy Gate	
• classify_request_safety(prompt) • detect_pi(prompt) • detect_destructive_intent(prompt)	• Reject direct identifying requests • Reject destructive/operational database/SQL requests • Reject raw PHI/PII grabs • Only allow analytical read only requests
Edge 2: Policy gate → Intent Classifier	
• classify_intent(prompt) • some kind of tool to extract information about query...tbd	• Don't infer if vague, ask for clarification
Edge 3: Intent Classifier → Clarification Node	
• detect_ambiguity(intent) • generate_clarifying_question(intent)	• ask for clarification if extracted information is not clear • Do not attempt to retrieve or query data if ambiguous

	• MUST return to Intent Classifier and loop through, until output of intent classifier is unambiguous
Edge 4: Intent Classifier → Retrieval Permission Gate	
• <code>get_user_permissions(user)</code> • <code>get_allowed_tables(__)</code> • <code>get_allowed_columns(__)</code> • <code>get_allowed_documents(__)</code>	• Restrict retrieval based on user, whether it's an epic source • Don't expose unnecessary details • Exclude sensitive columns unless authorized
Edge 5: Retrieval Permission Gate → Retrieval Layer	
• <code>build_retrieval_query(intent)</code> • <code>retrieval_scope(intent, permissions)</code> • <code>mask_columns(permissions)</code>	• Retrieval query cannot include forbidden data • Scope cannot be broadened beyond original intent • Mask any sensitive data to prevent leakage to LLM
Edge 6: Retrieval Layer → Retrieved Context Gate	
• <code>lexical_search(query)</code> • <code>vector_search(query)</code> • <code>schema_graph_search(query)</code> • <code>get_semantic_def(query)</code> • <code>rank_retrieved_context(results)</code>	• Treat all context as unverified • Deduplicate conflicting docs • Track source documents to cite later • Return bounded context, not entire document • Mark context confidence and source reliability
Edge 7: Retrieved Context Gate → Query Plan AST	
• <code>score_context_relevance(context, prompt)</code> • <code>package_retrieval(context)</code> • <code>select_candidate_tables(context, permissions)</code>	• Ensure retrieved tables are relevant • Ensure required columns and joins are present • Do not continue to query plan if context is weak

• select_candidate_columns(context, permissions) • check_join_paths_present(context) • attach_citations(retrieval) • validate_context_permissions(context, permissions)	• Exclude sensitive columns or information within retrieval
Edge 8 : Query Plan AST → Plan Safety Gate	
• build_query_plan(question, intent, context) • construct_plan(plan)	• Plan cannot introduce new objectives not in original request • Use semantic definitions
Edge 9: Plan Safety Gate → SQL/Python	
• validate_plan_permissions(plan, permissions) • check_sensitive_fields(plan) • check_output(plan) • estimate_plan_complexity(plan)	• Permissions (approved tables/columns only, no PHI/PII, no specific patient information retrieval) • Complexity is reasonable
Edge 10: SQL/Python → SQL Static Validation	
• generate_sql_from_plan(plan) • generate_python_from_plan(plan) (if needed) • maybe a tool to attach logs	• LLM only received approved plan and context • No filesystem access • No shell • No network • No secrets • No database writes • No agent spawning
Edge 11: SQL Static Validation → BigQuery Dry Run	
• parse_sql(sql) • reject_write_ops(sql) • check_forbidden_tables(sql, permissions)	• Reject SQL ops listed above • Reject SELECT * • Reject forbidden tables/columns • Reject SQL that doesn't do what the plan described

• check_forbidden_columns(sql, permissions) • check_sql_matches_plan(sql, plan)	
Edge 12: BigQuery Dry Run → Cost + Execution Gate	
• dry run big query • estimate bytes • estimate runtime • accessed tables	• dry run only • confirm query validity • confirm actually accessed tables match plan • record estimated cost
Edge 13: Cost + Execution Gate → Read Only Query Execution	
• check cost limit • check runtime limit • final permissions check • log record • approve execution	• Check if cost is below threshold • Check runtime below threshold • Read-only, etc other permissions enforced • Verify all permissions • Logs • If failed, loop back to Query Plan AST or SQL generation without adding new permissions, then run through
Edge 14: Read-Only Query Execution → Result Safety Gate	
• run read only query • enforce timeout •	• Don't expose secrets to LLM • ReadOnly credentials • Return only raw result to Result Safety gate
Edge 15: Result Safety Gate → Intepretation & Citations	
• scan result for phi/pii • detect personal identifiers • check result matches the plan • redact disallowed results	• Ensure patient safety by redacting names, dob, address, contact info, etc • Ensure result matches request • Reject/redact disallowed outputs

Edge 16: Interpretation & Citations → Final	
• summarize result • explain • cite • limitations	• include assumptions and limitations • cite sources •
Bounded Follow-Up controller → Pipeline Re-entry	
• classify followup • select reentry node(session_id) • reuse approved context(session_id) •	• enforce maximum retry limit • no new permissions, tools, agent spawning, access • re-enter from latest correct node, and pass following validation gates

Failure Points and their corresponding Re-Entry Nodes

Immediate Termination

- Insufficient Permissions
- Attempt to Access Prohibited Data
- Destructive Request
-

Failure Category	Example	Re-entry Node	Retry Limit
User Input	Ambiguous question, missing information	Clarification Node → Intent Classifier	Depends on user response
Retrieval	Missing tables, missing columns, poor retrieval relevance, missing join paths	Retrieval Layer	

Query Planning	Invalid plan, forbidden tables/columns, too costly	Query Plan AST	
Code Generation	Invalid SQL/Python, syntax errors, code execution doesn't match plan	SQL/Python Generation	
Execution Validation	Dry run fails, excessive cost, timeout, unexpected tables accessed	Query Plan AST	
Execution	BigQuery error	Read-Only Query Execution	
Output Safety	PHI/PII detected, result doesn't match approved request	Result Safety Gate	